

Gestione degli Errori

Gestione degli Errori

Come gestire gli errori di base in C++ senza eccezioni avanzate.

Cosa succede quando il programma va in errore?

Tutti i programmi possono ricevere input sbagliati, trovare file mancanti o fare operazioni impossibili (come dividere per zero). Un programma che "esplode" senza spiegazioni è inutilizzabile.

La **gestione degli errori** è l'arte di intercettare queste situazioni e rispondere in modo sensato, senza far crashare il programma.

I tre tipi di errori

- **Errori di compilazione:** il codice non è scritto correttamente. Il compilatore li trova prima ancora di eseguire il programma (es. hai dimenticato un `;`).
- **Errori di runtime:** il codice è scritto bene, ma succede qualcosa di sbagliato durante l'esecuzione (es. divisione per zero, file non trovato).
- **Errori logici:** il programma gira senza crashare, ma produce risultati sbagliati. Sono i più difficili da trovare.

Gestione semplice: codici di ritorno

Un approccio comune è far restituire alle funzioni `true` se tutto è andato bene, `false` se c'è stato un errore:

```

// La funzione restituisce true se riesce, false se c'è un errore
bool dividi(int a, int b, int& risultato) {
    if (b == 0) {
        return false;    // errore: non si può dividere per zero
    }
    risultato = a / b;
    return true;
}

int main() {
    int risultato;

    if (dividi(10, 2, risultato)) {
        cout << "Risultato: " << risultato << endl;    // 5
    } else {
        cout << "Errore: divisione per zero!" << endl;
    }

    if (dividi(10, 0, risultato)) {
        cout << "Risultato: " << risultato << endl;
    } else {
        cout << "Errore: divisione per zero!" << endl;    // questo viene
stampato
    }

    return 0;
}

```

Validare l'input dell'utente

L'errore più comune è ricevere dati non validi dall'utente. Controlla sempre prima di usare:

```
int leggiEtaValida() {
    int eta;
    do {
        cout << "Inserisci la tua eta (0-120): ";
        cin >> eta;
        if (eta < 0 || eta > 120) {
            cout << "Eta non valida. Riprova." << endl;
        }
    } while (eta < 0 || eta > 120);
    return eta;
}
```

Controllare se `cin` ha letto correttamente

Se l'utente inserisce una lettera dove ti aspetti un numero, `cin` va in errore. Puoi verificarlo:

```
int numero;
cout << "Inserisci un numero: ";

if (cin >> numero) {
    cout << "Hai inserito: " << numero << endl;
} else {
    cout << "Errore: non hai inserito un numero valido." << endl;
    cin.clear(); // resetta lo stato di errore di cin
    cin.ignore(1000, '\n'); // scarta il testo non valido
}
```

Le eccezioni: per errori gravi

Le **eccezioni** sono il meccanismo del C++ per gestire errori che interrompono il flusso normale. Funzionano con tre parole chiave: `try`, `catch`, `throw`.

- `throw` lancia un errore
- `try` racchiude il codice da monitorare
- `catch` intercetta l'errore e gestisce la situazione

```

#include <stdexcept>
using namespace std;

double dividi(double a, double b) {
    if (b == 0) {
        throw runtime_error("Divisione per zero!"); // lancia l'errore
    }
    return a / b;
}

int main() {
    try {
        cout << dividi(10, 2) << endl; // 5 - va bene
        cout << dividi(10, 0) << endl; // lancia l'eccezione
    } catch (runtime_error& e) {
        // Il programma non crasha - gestiamo l'errore qui
        cout << "Errore catturato: " << e.what() << endl;
    }

    cout << "Il programma continua normalmente." << endl;
    return 0;
}

```

Tipi di eccezioni già pronti

La libreria `<stdexcept>` fornisce tipi di errore già pronti da usare:

```

#include <stdexcept>

throw runtime_error("Errore generico durante l'esecuzione");
throw invalid_argument("L'argomento passato non è valido");
throw out_of_range("Il valore è fuori dall'intervallo consentito");

```

Esempio pratico: radice quadrata sicura

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

// Calcola la radice quadrata, ma solo per numeri non negativi
double radiceQuadrata(double x) {
    if (x < 0) {
        throw invalid_argument("Impossibile calcolare la radice di un
numero negativo");
    }
    return sqrt(x);
}

int main() {
    double valori[] = {16.0, -4.0, 25.0, 0.0};

    for (double v : valori) {
        try {
            cout << "sqrt(" << v << ") = " << radiceQuadrata(v) << endl;
        } catch (invalid_argument& e) {
            cout << "Errore per " << v << ": " << e.what() << endl;
        }
    }

    return 0;
}
```

Output:

```
sqrt(16) = 4
Errore per -4: Impossibile calcolare la radice di un numero negativo
sqrt(25) = 5
sqrt(0) = 0
```

Controllare l'apertura di un file

Prima di leggere o scrivere un file, verifica sempre che sia stato aperto:

```
#include <fstream>

ifstream file("dati.txt");

if (!file.is_open()) {
    cerr << "Errore: impossibile aprire il file 'dati.txt'" << endl;
    return 1;    // restituisce 1 per indicare che c'è stato un errore
}

// Qui sei sicuro che il file è aperto correttamente
```

`cerr` funziona come `cout`, ma manda il messaggio al canale degli errori — utile per distinguere i messaggi di errore dall'output normale.

Regole pratiche

- Valida sempre l'input dell'utente prima di usarlo
- Controlla sempre il risultato delle operazioni che possono fallire (apertura file, divisioni, ecc.)
- Non ignorare mai un errore in silenzio — il programma sembrerà funzionare ma darà risultati sbagliati
- Scrivi messaggi di errore chiari: l'utente deve capire cosa è andato storto
- Usa `cerr` per i messaggi di errore, non `cout`